

OMG Governance Documentation Package

The ten mandatory deliverables of the OrchestrAI Delivery Framework (ODF v1.0), covering OrchestrAI Model Governance (OMG) as of Core Platform v1.0 Complete — Releases 1 through 10 plus the Final Strategic Positioning update. Every figure in this package is traceable to a file in the platform's own repository, and every known limitation is stated rather than omitted.

● 9 governance domains

● 56 modules

● 44 data models

● 85 API endpoints

● 0% test & CI maturity

10

Mandatory deliverables

10

Releases delivered

7

Governance roles

Fair

Overall readiness rating

How to read this package. Documents 06, 07 and 09 state open gaps directly — including that server-side authentication is not yet implemented and no automated test suite exists. These are recorded as current limitations with a stated remediation sequence, not implied away.

Executive Solution Blueprint

Business problem, strategic objectives, scope, stakeholders, success criteria

1. Business Problem & Context

Enterprises deploying AI applications, autonomous agents, LLMs, copilots, RAG systems and third-party AI services face a governance gap that spreadsheets and email cannot close. Leadership cannot reliably answer, on demand:

- *"What AI does the organisation actually run?"* — there is no single, canonical inventory.
- *"Who owns each system, and who is accountable when it fails?"* — ownership lives in tribal memory, not a record.
- *"Has it been independently validated, and is the evidence retrievable?"* — evidence is scattered across drives and inboxes.
- *"Was production use formally approved, by whom, and why?"* — approvals live in email threads with no durable trail.
- *"Can we prove, to a regulator or a board, that this AI is under control?"* — assembling that proof today takes weeks, not minutes.
- *"Can our governance model be adapted to our own regulatory and industry context without a platform rebuild?"* — most governance tooling is either generic checklists or a bespoke one-off build.

OMG closes this gap with a single system of record for AI governance, and — as of Release 10 — a configuration layer that lets the governance logic itself be tuned per customer without code changes.

2. Strategic Objectives

OMG delivers a centralized AI governance operating platform that enforces:

1. **A single AI inventory.** Every AI asset the organisation runs — built or bought — recorded once, typed across 9 asset classes.
2. **Named, five-way accountability.** Every asset carries a Business Owner, Technical Owner, Risk Owner, Compliance Owner and Approver.
3. **Independent validation and evidence.** Six validation disciplines and ten mandatory ODF deliverable types, versioned and attributed.
4. **A deterministic decision layer.** A five-pillar governance readiness score drives a formal GO / CONDITIONAL GO / NO GO decision, never left to informal judgement.
5. **Governance that reasons, not just records.** Policy → Condition → Violation → Finding → Outcome → Recommended Action, computed live from real state, with every outcome explainable.
6. **End-to-end decision traceability.** Any governance decision can be reconstructed from the regulatory source to the human who acted on it.
7. **Configuration, not rebuilds.** The Governance Intelligence Studio lets a governance admin edit which conditions, outcomes and actions are active — tuning the platform to a customer without redeploying code.
8. **Continuous governance after approval.** Health monitoring, alerts, corrective actions, human override and emergency suspension keep the governance position current after go-live, and material change forces reassessment.
9. **A durable audit trail.** Every governance action is written to an append-only log, reconstructable on demand.

3. Project Scope

The current platform baseline (Core Platform v1.0) spans nine governance domains and fifty-six modules, delivered across ten numbered releases plus a final strategic positioning update:

DOMAIN	ANSWERS	MODULES
AI Governance Registry	What AI exists?	4
Executive Governance	Is enterprise AI under control?	5

DOMAIN	ANSWERS	MODULES
Policy Governance	What rules govern our AI?	3
Risk, Compliance & Intelligence	Can this AI be trusted?	19
Decision Governance	Can this AI move?	5
Change Governance	What changed, and who approved it?	5
Operations & Monitoring	What is happening now?	8
Audit & Oversight	Prove what happened, and why.	4
Administration	Who can do what?	3

The "Risk, Compliance & Intelligence" domain carries the platform's most recent growth: the Compliance Pack Framework, the Regulatory Knowledge Engine, the Governance Intelligence Engine, the Governance Actions Engine, Governance Decision Traceability, and the Governance Intelligence Studio — delivered in that order across Releases 5 through 10.

Out of scope for this baseline (see 09_Production_Readiness_Assessment.md for the full gap list): cryptographically verified authentication, multi-tenancy, an automated test suite and CI pipeline, and pre-loaded content for named regulatory frameworks beyond the platform's own regulatory-knowledge mechanism (ISO 42001, the EU AI Act, NIST AI RMF and a fuller RBI mapping are configured through the platform, not yet loaded as data).

4. Stakeholders & Accountability Model

OMG does not assume a single fixed organisation chart — accountability is enforced through seven role types, defined in the Prisma `UserRole` enum and mirrored by the frontend's route-level RBAC:

ROLE	MANDATE
Super Admin	Full platform administration, including RBAC configuration
Governance Admin	Runs the governance operating model — policies, conditions, outcomes, actions

ROLE	MANDATE
Risk Officer	Owns risk classification and risk-driven escalation
Business Owner	Accountable for the asset's business outcome and residual risk
Validator	Independent reviewer across the six validation disciplines
Auditor	Read access to the full audit trail and reporting surfaces
Viewer	Read-only visibility for stakeholders outside the governance team

Each asset additionally carries five named accountable owner fields (Business, Technical, Risk, Compliance, Approver) — see `02_Functional_Requirements_Specification.md`. The demo tenant seeds one illustrative persona per role so the model can be exercised end-to-end; a production deployment replaces these with the customer's own named individuals.

5. Success Criteria & Key Metrics

- **Governance coverage:** every registered AI asset carries a complete five-role ownership record and a computed governance readiness score — never a percentage entered by hand.
- **Reasoning transparency:** every governance Outcome ships with its own explicit reasons, and every recorded human decision can be reconstructed end-to-end via Decision Traceability.
- **Configuration over rebuild:** a governance admin can enable/disable a condition type, an outcome tier, or edit the action recommendation library through the Governance Intelligence Studio, with the change taking effect on the next read — no deployment required.
- **Audit readiness:** Board & Regulator reporting and the Decision Evidence Pack are generated on demand from live data, not assembled retrospectively.
- **Production persistence:** governance records for the domains delivered since Release 4 are written live to a shared Postgres system of record (Neon), not held only in browser storage — see `03_Solution_Architecture_Blueprint.md` for the one still-local exception (the audit log itself).

6. Expected Outcomes

OMG gives an organisation one place to see whether its AI estate is under control: a typed inventory with named accountability, a deterministic decision layer instead of ad hoc approval, governance reasoning the organisation can see and explain, a reconstructable trail for any decision, and a configuration mechanism — the Governance Intelligence Studio — that lets the same platform serve different regulatory and industry contexts (Banking, Insurance, Healthcare, Government, Enterprise) without rebuilding it per customer.

Read alongside 24_Final_Strategic_Blueprint_Design_Partner_Program.md for the platform's forward-looking positioning and the Founding Governance Partners engagement model.

Functional Requirements Specification

Features by domain, user stories, acceptance criteria, business rules

1. Feature Specifications by Governance Domain

AI Governance Registry

- Centralized registry supporting 9 AI asset types: Application, Agent, Model, LLM, Copilot, RAG System, AI Workflow, Multi-Agent System, Third-Party AI Service.
- Asset metadata, versioning, department ownership, tech stack tags and data sensitivity classification.
- A derived, eight-stage lifecycle position (AI Asset → Ownership → Risk → Validation → Evidence → Decision → Production → Monitoring) computed from live governance state, not stored as a status field.

Executive Governance

- Command Center and Executive Hub with four role-tuned lenses (CIO/CTO, CRO, Compliance, Board).
- Governance scorecards, executive heatmaps, governance insights, and Board & Regulator reporting generated from the live record.

Policy Governance

- A policy registry with seven automatic policy-breach detection rules evaluated continuously against live governance state.
- Policy-to-asset mapping and a policy violation tracker.

Risk, Compliance & Intelligence

- **Risk classification:** a five-dimension risk assessment (context, data sensitivity, decision impact, operational impact, control oversight) producing one of four risk tiers (Low, Medium, High, Critical).

- **Validation:** independent review across six disciplines (Business, Technical, Security, Compliance, Operational, Model), with a findings tracker running Open → In Progress → Resolved → Verified.
- **Evidence:** ten mandatory ODF deliverable types per asset, each versioned, attributed and status-tracked.
- **Compliance Pack Framework** (Release 5/5.1): Pack → Requirement → Control → Evidence, coverage computed from real linkage rather than a hand-entered score.
- **Regulatory Knowledge Engine** (Release 6): Source → Requirement → Obligation → Control → Evidence, the reusable foundation any future regulation configures onto.
- **Governance Intelligence Engine** (Release 7): Policy → Condition → Violation → Finding → Outcome, computed live, with every Outcome carrying its own explicit reasons.
- **Governance Actions Engine** (Release 8): Outcome/Finding → Recommended Action, with a human Accept / Reject / Defer decision layer — nothing executes automatically.
- **Governance Decision Traceability** (Release 9): full chain reconstruction (Condition → Policy → Violation → Finding → Outcome → Action → Human Decision) plus a printable Decision Evidence Pack.
- **Governance Intelligence Studio** (Release 10): a Condition Designer, Outcome Designer and Action Designer that make the reasoning engine's configuration editable at runtime, a Rule Mapping Engine visualization, a Compliance Pack Builder visualization, and Customer Governance Profiles (Banking, Insurance, Healthcare, Government, Enterprise) with exactly one active per tenant.

Decision Governance

- A five-pillar governance readiness score (Ownership 25%, Risk 20%, Validation 20%, Decision 20%, Evidence 15%) computed out of 100.
- A governance blockers evaluator surfacing any failing pillar as an explicit blocker.
- Decision Authority workbench recording a formal outcome — GO, CONDITIONAL GO, NO GO, or PENDING — with a named decision owner and justification.

Change Governance

- Change classification across six categories (Model, Vendor, Data, Prompt, Policy, Operational).
- Impact assessment across seven governance areas, producing a weighted 0–100 impact score.
- A reassessment rules engine resolving the score to a change magnitude (Minor / Moderate / Major / Critical), each routing to a different approval depth.
- A ten-state governance state machine (Draft, Review, Approved, Production, Monitoring, Change Requested, Impact Assessment, Reassessment, Reapproved, Retirement) with six trigger rules that convert a detected change condition into governance work automatically.

Operations & Monitoring

- Continuous governance health scoring across five pillars (ownership, risk, validation, compliance, operational), rolling up to Healthy / Watchlist / Attention Required.
- Emergency Kill Switch console, Human Override tracker, Governance Incident management, Corrective Actions, and a Scheduled Review calendar.
- Executive alerting derived live from governance state (Critical Risk, Missing Ownership, Policy Violation, Expired Review).

Audit & Oversight

- An append-only audit log capturing who did what, when, and why, across every governance action.
- Four reconstruction views: Governance Timeline, Change History, Board & Regulator Reporting, and Decision Traceability.

Administration

- User management (create, edit role, activate/deactivate).
 - An RBAC Administration screen rendering the live role-to-module authorisation matrix, generated from the same navigation configuration the product itself uses.
-

2. Representative User Stories & Acceptance Criteria

US-01 — Register an AI asset. *As a Business Owner, I want to register a new AI system with its type, department and intended purpose, so that it enters the governed inventory before anyone starts using it.* - **AC-01a:** The asset cannot progress past Draft status until it is typed and named. - **AC-01b:** The asset appears immediately in the AI Governance Registry and in the derived eight-stage journey view.

US-02 — Assign accountable ownership. *As a Governance Admin, I want to name a Business, Technical, Risk and Compliance Owner and an Approver for an asset, so that there is always someone accountable.* - **AC-02a:** Ownership completeness is scored and weighted 25% of the readiness score — the single heaviest pillar. - **AC-02b:** A missing mandatory role raises a "Missing Owner" governance condition automatically.

US-03 — Move an asset to production. *As a Decision Authority, I want to see a deterministic readiness score and any blockers before recording GO / CONDITIONAL GO / NO GO, so that the decision is informed, not arbitrary.* - **AC-03a:** Any failing pillar becomes an explicit governance blocker that must be cleared or accepted before a GO. - **AC-03b:** The decision is written to the audit log with the decision owner and justification.

US-04 — Configure governance logic for a new customer. *As a Governance Admin, I want to enable or disable a condition type, an outcome tier, or edit an action's recommendation text, so that OMG's reasoning matches our own governance model without a code change.* - **AC-04a:** A disabled condition type is never raised, for any asset, from the next read onward. - **AC-04b:** A disabled outcome tier is skipped by the reasoning engine, which falls through to the next enabled tier in the fixed escalation order.

US-05 — Reconstruct a governance decision. *As an Auditor, I want to replay the full reasoning chain behind any recorded decision — from the regulatory obligation through to who accepted the recommended action — so that I can produce audit-ready evidence on demand.* - **AC-05a:** The Timeline view renders the chain in causal order (Condition → Policy → Violation → Finding → Outcome → Action → Decision), not wall-clock order. - **AC-05b:** A Decision Evidence Pack can be generated as a printable, structured document for any traced asset.

3. Business Rules

- **BR-1:** High and Critical risk assets require an approved independent validation before production (policy rule `REQUIRE_VALIDATION_FOR_HIGH_RISK`).
 - **BR-2:** An asset is evidence-complete at three or more approved deliverables; below that threshold a production asset raises policy rule `REQUIRE_EVIDENCE_FOR_PRODUCTION` .
 - **BR-3:** Reassessment magnitude is resolved from the impact score: Critical ≥ 70 (Executive Approval Required), Major ≥ 45 (Full Governance Review), Moderate ≥ 20 (Risk Review Required), Minor ≥ 0 (No Reassessment).
 - **BR-4:** Governance readiness score weights are fixed at Ownership 25%, Risk 20%, Validation 20%, Decision 20%, Evidence 15% — always summing to 100.
 - **BR-5:** Governance Intelligence Studio condition and outcome detection *mechanisms* and the outcome escalation *order* are platform primitives and cannot be reordered through configuration — only which of them are switched on.
 - **BR-6:** Exactly one Customer Governance Profile is active per tenant at any time; activating one deactivates all others.
 - **BR-7:** Audit log entries are append-only by construction — no update or delete path exists for an audit record anywhere in the application code.
-

4. Assumptions

- A production deployment will replace the seeded demo personas with the customer's own named individuals in each of the seven roles.
- The customer accepts that role authorisation is currently enforced server-side by role claim (see `06_Security_Review_Document.md`) and that closing the authentication gap is a prerequisite for multi-user production use with untrusted clients.
- Regulatory content for named frameworks (ISO 42001, EU AI Act, NIST AI RMF, an expanded RBI mapping) will be configured through the Regulatory Knowledge Engine and the Governance Intelligence Studio as part of a customer engagement, not delivered pre-loaded in this baseline.

5. Constraints

- Single-tenant per deployment today — multi-tenancy is not implemented (see [09_Production_Readiness_Assessment.md](#)).
- No automated test suite or CI pipeline exists yet; verification to date has been manual, browser-based, and re-run against every release (see [07_Test_Strategy_And_Evidence.md](#)).
- Roughly 18 of the platform's 44 data models — spanning risk assessment, validation, decision, the earlier policy/change domains, and the operations console — do not yet have a corresponding write API and remain client-side only pending Release 11+ work.

1. System Architecture Overview

		DATA
TIER (Neon Serverless PostgreSQL)	44 models · 42 enums · 51 relations · 0 secondary indexes	AIAsset is the hub – 15+ child relations cascade on delete

2. Technology Stack

Frontend Application

- **Core framework:** React 19.2 + TypeScript 6.0.

- **Build tooling:** Vite 8.2, `@tailwindcss/vite` for utility styling.
- **Routing:** React Router DOM 7.18, 59 governed routes.
- **State management:** no external state library — three React Contexts (Auth, Theme, Experience) plus local component state.
- **Design system:** token-based (CSS custom properties in `tokens.css`), three themes — light, dark, glass.
- **Dependency surface:** deliberately small — only `react`, `react-dom` and `react-router-dom` as runtime dependencies. No UI kit, no chart library.

Backend API Server

- **Core framework:** NestJS 11.0 (Express HTTP adapter).
- **ORM:** Prisma Client 6.19.
- **Authorisation:** a single `RolesGuard` reading `@Roles(...)` metadata, applied per endpoint.
- **Footprint:** the entire backend is 6 files and 946 lines — the domain logic lives in the frontend's derivation layers, not the API.

Data Layer

- **Database:** Neon serverless PostgreSQL (managed via `DATABASE_URL`, never committed to the repository).
- **Schema management:** Prisma migrations (`prisma db push`), verified in sync with Neon after every release.

Hosting & Cloud Infrastructure

- **Frontend hosting:** Firebase Hosting.
- **Backend hosting:** Render.com Web Service.
- **Database:** Neon serverless PostgreSQL.
- **Source control:** GitHub (`skedcom-web/OrchestrAI-OMG`).

3. Frontend Derivation Layers

Governance state is computed, not typed in. Three layers sit between persistence and the UI:

1. **storageService.ts** — persistence orchestration and the Repository Pattern's cache-then-network logic (3,486 lines).
 2. **Reasoning and derivation engines** — `governanceReasoningEngine.ts`, `governanceActionsEngine.ts`, `decisionTraceabilityEngine.ts`, `regulatoryKnowledgeEngine.ts`, `compliancePackFramework.ts`, `readinessFoundation.ts` — pure, data-in/data-out functions introduced across Releases 1–10.
 3. **executiveGovernance.ts** — aggregation, scorecards and report generation (734 lines).
-

4. Integration & Persistence Design

- **Repository Pattern:** every governance domain since Release 6 is Api-first — the Local (localStorage) implementation exists only as a fallback utility, never selected by default.
 - **What is genuinely server-persisted:** roughly 26 of the platform's 44 data models — every domain from Release 4 onward (Assets, Evidence, Continuity, the Compliance Pack Framework, the Regulatory Knowledge Engine, Governance Intelligence, Governance Actions, and the Governance Intelligence Studio).
 - **What remains client-side only:** the audit log, plus the earlier Risk Assessment, Validation, Decision, Policy (Phase 9) and Change (Phase 10) and Operations domains — roughly 18 models — pending a write API (see `09_Production_Readiness_Assessment.md`).
 - **CORS:** an explicit allow-list (`CORS_ORIGINS` environment variable, defaulting to the production Firebase Hosting origin and localhost for development) — not an open policy.
-

5. Hosting Architecture

TIER	PROVIDER	NOTES
Frontend	Firebase Hosting	Static SPA build (<code>frontend/dist</code>), SPA rewrite to <code>index.html</code>
Backend API	Render.com	NestJS service, build <code>prisma generate && nest build</code> , start <code>node dist/main</code>
Database	Neon	Serverless PostgreSQL, connection string via <code>DATABASE_URL</code>

See 08_Deployment_Blueprint.md for the deployment workflow and rollback strategy.

Database Design Document

44 models, 12 domain clusters, table definitions, index strategy

1. Schema Overview

- **Database:** Neon serverless PostgreSQL, connected via `DATABASE_URL` (never committed to the repository).
- **Schema management:** Prisma ORM, `backend/prisma/schema.prisma` (1,345 lines), pushed with `prisma db push` and verified in sync with Neon after every release.
- **Scale:** 44 models, 42 enums, 51 relations, 0 secondary indexes (see §4).

This document lists every domain cluster and gives full field detail for the hub entity (`AIAsset`) and the four models introduced by the most recent release (the Governance Intelligence Studio). Every other model's field-level detail is authoritative in `backend/prisma/schema.prisma` itself — reproducing all 44 models' complete field lists here would risk drifting from the schema faster than this document could be kept current; the domain-cluster table below is the traceable index into it.

2. Domain Clusters (44 models)

CLUSTER	VINTAGE	MODELS	COUNT
Identity & Core	Baseline	User, AIAsset, AssetOwner	3
Assurance	Baseline	RiskAssessment, ValidationRecord, EvidenceDocument, Finding, ComplianceAssessmentRecord	5
Decision	Baseline	DecisionRecord	1
Operations	Baseline	KillSwitchRecord, OverrideRecord, GovernanceIncident, RetirementRecord,	7

CLUSTER	VINTAGE	MODELS	COUNT
		ScheduledReview, CorrectiveAction, GovernanceAlert	
Policy (Phase 9)	Baseline	Policy, PolicyMapping, PolicyViolation	3
Change (Phase 10)	Baseline	ChangeRequest, ChangeApproval, StateTransition, ChangeHistoryEntry, GovernanceTriggerRule	5
Continuity & Evidence	Releases 3–4	EvidenceRecord, ReassessmentTrigger, GovernanceReauthorizationRecord	3
Compliance Pack Framework	Release 5 / 5.1	CompliancePack, ComplianceRequirement, PackControl, EvidenceMapping	4
Regulatory Knowledge Engine	Release 6	RegulatorySource, RegulatoryRequirement, Obligation, ObligationControl, ObligationEvidenceMapping	5
Governance Intelligence	Releases 7–9	GovernancePolicy, GovernanceFinding, RecommendedAction	3
Governance Intelligence Studio	Release 10	ConditionDefinition, OutcomeRule, ActionRule, GovernanceProfile	4
Audit	Baseline	AuditLog	1

AIAsset is the hub: 15+ child relations cascade on delete. Every governance artefact — validation, evidence, decision, incident, policy violation, change request, condition, outcome, recommended action — hangs off the asset.

3. Detailed Table Definitions

AIAsset (hub entity)

Core fields (a non-exhaustive selection — see schema.prisma for the complete set, which also carries the Release 1 Governance Authority Profile fields):

FIELD	TYPE	NOTES
id	UUID (PK)	@default(uuid())

FIELD	TYPE	NOTES
name	String	
type	Enum <code>AssetType</code>	Application, Agent, Model, LLM, Copilot, RAG_System, AI_Workflow, Multi_Agent_System, Third_Party_Service
description	Text	
department	String	
version	String	default <code>"1.0.0"</code>
status	Enum <code>GovernanceStatus</code>	default <code>DRAFT</code>
operationalStatus	String	default <code>"Active"</code>
riskLevel	Enum <code>RiskLevel</code>	default <code>MEDIUM</code>
techStack	String[]	
dataSensitivity	String	default <code>"Confidential"</code>
validationScore	Int?	nullable
decisionOutcome	Enum <code>DecisionOutcome</code>	default <code>PENDING</code>
tags	String[]	
accountableOwner , governanceSponsor , humanOverrideAuthority , killSwitchAuthority , reassessmentAuthority	String?	Release 1 Governance Authority Profile
authorityRiskOwner , authorityTechnicalOwner , authorityComplianceOwner	String?	Release 4 — API-writable Authority Profile scalars
ownershipJson	Json?	Release 4 — the free-text five-role Ownership Matrix
oversightType	Enum <code>HumanOversightType</code> ?	Release 1 — Human Oversight Classification

FIELD	TYPE	NOTES
<code>autonomyLevel</code>	Int?	Release 1 — Autonomy Classification, Level 0–5
<code>createdAt</code> / <code>updatedAt</code>	DateTime	

ConditionDefinition (Release 10 — Condition Designer)

FIELD	TYPE	NOTES
<code>id</code>	String (PK)	
<code>conditionType</code>	Enum <code>GovernanceConditionType</code> , unique	one row per condition type
<code>label</code>	String	
<code>description</code>	Text	
<code>defaultSeverity</code>	Enum <code>GovernancePolicySeverity</code> , default <code>MEDIUM</code>	
<code>enabled</code>	Boolean, default <code>true</code>	gates whether the reasoning engine ever raises this condition type
<code>createdAt</code> / <code>updatedAt</code>	DateTime	

OutcomeRule (Release 10 — Outcome Designer)

FIELD	TYPE	NOTES
<code>id</code>	String (PK)	
<code>outcomeStatus</code>	Enum <code>GovernanceOutcomeStatus</code> , unique	one row per outcome tier
<code>description</code>	Text	
<code>enabled</code>	Boolean, default <code>true</code>	gates whether the reasoning engine can land on this tier
<code>createdAt</code> / <code>updatedAt</code>	DateTime	

ActionRule (Release 10 — Action Designer)

FIELD	TYPE	NOTES
<code>id</code>	String (PK), <code>@default(uuid())</code>	
<code>triggerType</code>	Enum <code>ActionRuleTriggerType</code>	<code>CONDITION</code> or <code>OUTCOME</code>
<code>triggerValue</code>	String	loose reference to a condition or outcome value
<code>actionType</code>	Enum <code>RecommendedActionType</code>	
<code>actionName</code>	String	
<code>actionDescription</code>	Text	
<code>enabled</code>	Boolean, default <code>true</code>	
<code>createdAt</code> / <code>updatedAt</code>	DateTime	
—	<code>@@unique([triggerType, triggerValue])</code>	one rule per trigger

GovernanceProfile (Release 10 — Customer Governance Profiles)

FIELD	TYPE	NOTES
<code>id</code>	String (PK)	
<code>name</code>	String	e.g. "Banking"
<code>industry</code>	String	
<code>description</code>	Text	
<code>isActive</code>	Boolean, default <code>false</code>	exactly one active row per tenant, enforced at the application layer
<code>createdAt</code> / <code>updatedAt</code>	DateTime	

4. Relationships, Keys & Index Strategy

- **Primary keys:** every model uses a string or UUID primary key; several Release 10 catalogue tables (`ConditionDefinition` , `OutcomeRule` , `GovernanceProfile`)

use a deterministic string `id` rather than a generated UUID, since they are seeded, fixed-cardinality catalogues.

- **Foreign keys:** 51 `@relation` declarations across the schema. `AIAsset` is the parent of the majority of them, with `onDelete: Cascade` on its child relations.
- **Composite uniques:** at least two — `AssetOwner.assetId` (one ownership row per asset) and `ActionRule.[triggerType, triggerValue]` (one rule per trigger).
- **Secondary indexes:** zero `@@index` or `@index` declarations exist in the schema today. This is a genuine, current gap, not a design choice — see `09_Production_Readiness_Assessment.md` for the remediation sequence (index every foreign key and common filter column before any volume load).

Source of truth for every field not reproduced above: `backend/prisma/schema.prisma`.

API Design Specification

85-endpoint REST directory, request/response models, authentication

1. API Surface Overview

A single NestJS controller (`app.controller.ts` , 785 lines) exposes **85 REST endpoints** under the `/api` prefix, organised by governance domain. All routes follow the same pattern — `GET` (list/detail), `POST` (create), `PATCH` (update), and `DELETE` where the domain supports deletion.

DOMAIN	BASE ROUTES	VERBS
Health	<code>/api/health</code>	GET
Assets	<code>/api/assets</code>	GET, POST, PATCH, DELETE
Evidence Records	<code>/api/evidence-records</code>	GET, POST, PATCH, DELETE
Continuity	<code>/api/reassessment-triggers</code> , <code>/api/reauthorization-records</code>	GET, POST, PATCH
Compliance Pack Framework	<code>/api/compliance-packs</code> , <code>/api/compliance-requirements</code> , <code>/api/pack-controls</code> , <code>/api/evidence-mappings</code>	GET, POST, PATCH, DELETE
Regulatory Knowledge Engine	<code>/api/regulatory-sources</code> , <code>/api/regulatory-requirements</code> , <code>/api/obligations</code> , <code>/api/obligation-controls</code> , <code>/api/obligation-evidence-mappings</code>	GET, POST, PATCH, DELETE
Governance Intelligence	<code>/api/governance-policies</code> , <code>/api/governance-findings</code> , <code>/api/recommended-actions</code>	GET, POST, PATCH, DELETE
Governance Intelligence Studio	<code>/api/condition-definitions</code> , <code>/api/outcome-rules</code> (no DELETE — fixed catalogues), <code>/api/action-rules</code> , <code>/api/governance-profiles</code> (no DELETE)	GET, POST, PATCH, (DELETE on action-rules only)

DOMAIN	BASE ROUTES	VERBS
Monitoring	<code>/api/monitoring/alerts</code> , <code>/api/monitoring/reviews</code> , <code>/api/monitoring/corrective-actions</code>	GET, POST, PATCH
Administration	<code>/api/users</code>	GET
Audit	<code>/api/audit-logs</code>	GET

Not yet exposed: write endpoints for RiskAssessment, ValidationRecord, DecisionRecord, the Phase 9 Policy domain, the Phase 10 Change domain, and the Operations console (KillSwitchRecord, OverrideRecord, GovernanceIncident, RetirementRecord) — these remain client-side only pending future release work.

2. Request & Response Models

Every domain follows the same envelope shape. Example — Governance Intelligence Studio's `ConditionDefinition` :

GET `/api/condition-definitions`

```
[ { "id": "cond-evidence-expired", "conditionType": "EVIDENCE_EXPIRED",
  "label": "Evidence Expired", "description": "One or more evidence records
  supporting this asset have passed their expiry date.", "defaultSeverity":
  "HIGH", "enabled": true, "createdAt": "2026-08-24T00:00:00.000Z",
  "updatedAt": "2026-08-24T00:00:00.000Z" } ]
```

PATCH `/api/condition-definitions/:id` — request body accepts any subset of writable fields (e.g. `{ "enabled": false }`); response is the updated row.

Enum values are transmitted in their backend `UPPER_SNAKE_CASE` form (matching the Prisma schema) and translated to frontend-readable strings (e.g. `"Evidence Expired"`) by a dedicated `enumMaps.ts` mapping layer in the frontend Repository Pattern — the API itself is enum-canonical, not string-loose.

3. Authentication & Authorisation Requirements

- **Authorisation:** every data-bearing endpoint declares required roles via `@Roles(...)` metadata, enforced by `RolesGuard`. The guard fails closed — a missing, malformed or unrecognised role claim is denied (403), and there is no implicit super-user bypass.
 - **Role claim transport:** the caller's role is supplied via the `x-user-role` request header.
 - **Authentication: not yet implemented.** The role claim is not cryptographically verified — a caller can currently assert any role via the header. This is authorisation, not authentication, and is documented as the highest-priority open item in `06_Security_Review_Document.md` and `09_Production_Readiness_Assessment.md`.
 - **CORS:** an explicit origin allow-list (`CORS_ORIGINS` environment variable), not an open policy.
-

4. Error Handling

Standard NestJS HTTP exception shape, returned as JSON:

Missing role claim

```
{ "statusCode": 403, "message": "A role claim is required for this endpoint. Supply the x-user-role header.", "error": "Forbidden" }
```

Unrecognised role

```
{ "statusCode": 403, "message": "Unrecognised role claim.", "error": "Forbidden" }
```

Role not authorised for this endpoint

```
{ "statusCode": 403, "message": "Your role is not authorised for this endpoint.", "error": "Forbidden" }
```

Every denial is logged server-side (`RolesGuard` 's internal logger) with the method, URL and the claimed role, before the 403 is returned.

Source of truth: `backend/src/app.controller.ts` (route declarations),
`backend/src/auth/roles.guard.ts` (authorisation logic).

Security Review Document

Authentication model, authorisation model, data protection, risk register

1. Authentication Model

Current state: authorisation only, not authentication. There is no password field, credential store, or session/token issuance anywhere in the codebase. The frontend's persona switcher selects a role for the demo experience; the backend trusts whatever role is presented in the `x-user-role` request header.

This is a deliberate, documented gap rather than an oversight — it is the top-ranked item in both the 09_Production_Readiness_Assessment.md and the technical roadmap. **A caller who can reach the API can currently assert any role.** Closing this gap requires credential verification issuing a signed session or JWT, with the role resolved server-side from the authenticated user record rather than trusted from a header — before the platform is exposed to untrusted clients or multiple real users sharing one deployment.

2. Authorisation Model

- **Mechanism:** NestJS `RolesGuard`, applied via `@Roles(...)` decorator metadata on every data-bearing endpoint across all 85 API routes.
- **Fail-closed design:** a missing role header, a role claim outside the Prisma `UserRole` enum, or a role not listed in the endpoint's required roles is denied with HTTP 403 — the guard never admits by default.
- **No super-user bypass:** `SUPER_ADMIN` is granted access only where an endpoint's `@Roles(...)` list explicitly includes it, exactly like every other role — there is no implicit "admin sees everything" shortcut in the guard itself.
- **Denial logging:** every denied request is logged server-side with the method, URL and the claimed role.

- **Client-side layer:** `navigation.ts` declares every route as an RBAC key; each persona carries an `allowedNav` list, enforced at the route boundary by `ProtectedRoute`. This is a UX-layer guard (it prevents accidental navigation) and is not a substitute for server-side enforcement — the client can be bypassed by calling the API directly, which is exactly why the server-side `RolesGuard` exists independently.
 - **Seven role types:** `SUPER_ADMIN`, `GOVERNANCE_ADMIN`, `RISK_OFFICER`, `BUSINESS_OWNER`, `VALIDATOR`, `AUDITOR`, `VIEWER` — defined once in the `Prisma UserRole` enum and consumed by both the guard and the frontend's navigation config, so the two cannot drift apart.
-

3. Data Protection

- **Transport:** all traffic to Firebase Hosting, Render, and Neon is HTTPS/TLS by default on each managed platform.
 - **Database connection:** Neon's connection string is supplied exclusively via the `DATABASE_URL` environment variable and is never committed to source control.
 - **CORS:** an explicit origin allow-list (`CORS_ORIGINS` environment variable, defaulting to the production Hosting origin and localhost for development), not an open `*` policy.
 - **Data sensitivity classification:** every AI asset carries a `dataSensitivity` field (default `"Confidential"`), letting the registry distinguish PII/sensitive assets from general internal ones.
-

4. Secrets Handling

- `.env` and `.env.local` are excluded from source control via `.gitignore`.
- No hardcoded credentials, API keys, or connection strings exist in the application source (`backend/src`, `frontend/src`).
- Backend runtime secrets (`DATABASE_URL`, `CORS_ORIGINS`, `PORT`) are read exclusively from `process.env` at startup.

- Frontend build-time configuration (`VITE_API_URL`) is read via `import.meta.env` , defaulting to the production Render URL when unset.

5. Security Risks & Mitigation Strategy

RISK	CURRENT MITIGATION	RESIDUAL EXPOSURE
Unauthenticated role assertion	RolesGuard fails closed on every endpoint; denials logged	The role claim itself is not cryptographically verified — see §1
Cross-origin abuse	Explicit CORS allow-list, not open	Allow-list must be kept current as new frontend origins are added
Audit trail tampering	Audit entries are append-only by application convention — no update/delete code path exists	The audit log is still written to browser <code>localStorage</code> (see <code>03_Solution_Architecture_Blueprint.md</code>); a user with developer tools access could edit or clear local entries, and timestamps/IP are client-generated, not server-issued
Unindexed query surface	N/A — this is a performance/availability risk, not a confidentiality risk	Zero secondary database indexes exist today; see <code>09_Production_Readiness_Assessment.md</code>
Secret leakage	<code>.env</code> / <code>.env.local</code> gitignored; no hardcoded secrets found in source	Standard operational discipline (secret rotation, least-privilege DB credentials) remains an operator responsibility outside this codebase's scope

6. Priority Remediation

1. **Server-side authentication** — signed session or JWT, credential verification, role resolved from the authenticated user record. Closes the RolesGuard residual described in §1.
2. **Move audit log writes server-side** — server-issued timestamps, real client IP capture, and database-level append-only enforcement, closing the `localStorage` tamper surface described in §5.

3. **Database indexing** – index every foreign key and common filter column before any volume load, both for performance and to reduce the blast radius of slow/expensive queries.

Full sequencing and rationale: 09_Production_Readiness_Assessment.md.

Test Strategy & Evidence

Current test maturity, manual verification workflow, testing roadmap

1. Current State — Stated Plainly

No automated test suite exists today. There is no test runner configured, no `.test.` or `.spec.` files in the frontend or backend application code, and no CI pipeline. This is recorded here in full rather than implied away, consistent with the ODF principle that a governance deliverable must be honest about gaps.

What has substituted for automated testing to date is a **manual, browser-based verification workflow, repeated for every release:**

1. **Compile-gate verification.** Every release is required to pass `tsc --noEmit` (strict TypeScript) on both `frontend/` and `backend/` before deployment, and `npm run build` must complete cleanly.
2. **Schema-sync verification.** `prisma db push` is run against live Neon after every schema change, and the CLI's own diff output (an empty migration) is treated as the pass condition.
3. **Live browser verification.** For every release touching the UI, a local backend is pointed at production Neon, the relevant page is exercised end-to-end in a real browser session (create/edit/toggle the new capability), and the resulting Neon state is inspected directly (via API calls) to confirm the mutation actually persisted — not just that the UI appeared to accept it.
4. **Cross-session/cross-device equivalence.** Because the frontend caches data in `localStorage` per browser origin, verification of "does this change apply everywhere" is performed by querying the live API directly (architecturally equivalent to a second browser or device), rather than only trusting a single browser tab's rendered state.
5. **State restoration.** Test mutations made during verification (toggling a condition, creating a scratch record) are reverted afterward so Neon is left in its seeded baseline state.

This process has been the actual quality gate for every one of the ten releases and the final positioning update documented in `docs/governance/`. It is real evidence of correctness for the specific changes it exercised, but it is **not** a substitute for a maintained, repeatable automated suite: it does not run unattended, does not prevent regression in code paths not touched by the current release, and depends on a human remembering to run it.

2. Unit Testing — Strategy (Not Yet Implemented)

Priority target when implemented: the reasoning and rules engines first, because they encode the governance logic itself and are pure, data-in/data-out functions — the cheapest and highest-value code to unit test in this codebase:

- `governanceReasoningEngine.ts` — `detectGovernanceConditions()`, `evaluatePolicyViolations()`, `computeGovernanceOutcome()`
- `governanceActionsEngine.ts` — `generateActionDrafts()`
- `decisionTraceabilityEngine.ts` — `buildDecisionTrace()`
- `readinessFoundation.ts` — the five-pillar scoring math
- `compliancePackFramework.ts` / `regulatoryKnowledgeEngine.ts` — coverage computation

3. Integration Testing — Strategy (Not Yet Implemented)

Target: the Repository Pattern's cache-then-network behaviour (optimistic cache update → API persist → reconciliation) and the NestJS controller's role-guard enforcement across all 85 endpoints — verifying that every data-bearing route actually declares and enforces its `@Roles(...)` list, not just the routes exercised manually to date.

4. System Testing — Strategy (Not Yet Implemented)

Target: end-to-end flows spanning the full governance chain — asset registration through ownership, risk, validation, evidence, decision, and into the Governance

Intelligence Engine's condition-detection and outcome-computation — run against a disposable database rather than production Neon.

5. UAT Strategy

To date, UAT has taken the form of the live browser verification described in §1, performed by the same party building the release. A production engagement should separate this: UAT performed by the customer's own governance team against a staging environment, following the acceptance criteria in 02_Functional_Requirements_Specification.md.

6. Regression Strategy — Strategy (Not Yet Implemented)

Without an automated suite, regression risk is currently managed only by the discipline of re-verifying the specific area touched by each release, plus the "computed, not stored" architectural choice (governance state is derived live rather than duplicated, reducing the number of places a regression can hide). A CI pipeline running the unit and integration suites above on every commit is the recommended next step — see 09_Production_Readiness_Assessment.md.

7. Test Maturity Summary

DIMENSION	STATUS
Compile-gate (<code>tsc --noEmit</code> , <code>npm run build</code>)	Enforced manually before every release
Schema-sync check (<code>prisma db push</code>)	Enforced manually before every release
Manual browser + live-Neon verification	Performed for every UI-facing release, not automated
Unit tests	None
Integration tests	None
System/E2E tests	None
CI pipeline	None

DIMENSION	STATUS
Test and CI maturity rating	0% — see 09_Production_Readiness_Assessment.md

Deployment Blueprint

Infrastructure, environment design, deployment workflow, rollback strategy

1. Infrastructure & Hosting

TIER	PROVIDER	LIVE ENDPOINT
Frontend Web App	Firebase Hosting	https://orchestrai-omg.web.app
Backend REST API	Render.com Web Service	https://orchestrai-omg.onrender.com
Database	Neon Serverless PostgreSQL	Connection via <code>DATABASE_URL</code> environment variable (not published in this document)
Source Control	GitHub	github.com/skedcom-web/OrchestrAI-OMG

2. Environment Design

- **Frontend build-time config:** `VITE_API_URL` — points the SPA at the backend API; defaults to the production Render URL when unset, and is overridden with a local `.env.local` during development/verification against a local backend.
- **Backend runtime config:** `DATABASE_URL` (Neon connection string), `CORS_ORIGINS` (comma-separated allow-list, defaults to the production Hosting origin plus localhost), `PORT` (defaults to 3000).
- No secrets are committed to source control; `.env` and `.env.local` are gitignored on both frontend and backend.

3. Deployment Workflow

Deployment is currently a **manual, developer-triggered process** — there is no CI/CD pipeline (see §5).

Frontend

```
cd frontend npm run build npx firebase deploy --only hosting
```

Backend schema changes

```
cd backend npx prisma format npx prisma db push
```

Render redeploys the backend service automatically on a push to the connected branch, running the build command `prisma generate && nest build` and the start command `node dist/main`.

Version control

```
git add <changed files> git commit -m "<description>" git push
```

4. Rollback Strategy

- **Frontend:** Firebase Hosting retains prior releases; a rollback is a one-click "roll back to this version" action in the Firebase Console, requiring no rebuild.
 - **Backend:** Render redeploys from the connected Git branch — a rollback is a `git revert` (or redeploying a prior commit) followed by a normal push, since there is no blue-green or canary mechanism configured.
 - **Database:** schema rollback is `git revert` on `schema.prisma` followed by `prisma db push`. Prisma's `db push` does not generate a reversible migration history — there is no automated data rollback, only schema rollback. This is a genuine gap for any change that also requires a data backfill or removal.
-

5. CI/CD — Current State

No automated CI/CD pipeline exists. There is no GitHub Actions workflow, no Dockerfile, and no infrastructure-as-code for either the Render service or the Firebase project — both are configured through their respective consoles. Every

deployment described in §3 is run manually by a developer, gated by the manual verification process in 07_Test_Strategy_And_Evidence.md, not by an automated pipeline.

This is recorded as an open item, not implied as already solved — see 09_Production_Readiness_Assessment.md.

Production Readiness Assessment

Readiness by dimension, full risk register, remediation sequence

1. Overall Assessment

Feature-complete, persistence-backed; not yet a deployable multi-user production system. The governance domain — the opinionated, hard-to-get-right part — is largely server-backed and coherent across ten releases. What remains before an untrusted, multi-user production deployment is conventional platform engineering: identity, the last mile of persistence, and operational hardening. This is a materially different, and materially better, position than the platform's earlier "browser-only, backend deployed but unused" baseline — see [03_Solution_Architecture_Blueprint.md](#).

2. Readiness by Dimension

DIMENSION	RATING
Functional completeness	96%
Domain model quality	92%
UX and accessibility	84%
Data architecture	78%
Authorisation design	72%
Security posture	38%
Operational readiness	25%
Test and CI maturity	0%

These ratings are a qualitative, evidence-grounded assessment (not a formula), consistent across 07_Test_Strategy_And_Evidence.md and 06_Security_Review_Document.md.

3. Production Readiness Checklist

- ☒ **Requirements completed** — see 02_Functional_Requirements_Specification.md
- ☒ **Architecture approved** — see 03_Solution_Architecture_Blueprint.md
- ☒ **Development completed** for the current baseline (9 domains, 56 modules, Releases 1–10)
- ☐ **Testing completed** — no automated suite exists; manual verification only (07_Test_Strategy_And_Evidence.md)
- ☒ **Security review completed** — RolesGuard hardened, fails closed, across all 85 endpoints; authentication remains open (06_Security_Review_Document.md)
- ☐ **Observability fully implemented** — audit logging exists but is client-side only; alerting is computed on read with no scheduler or notification channel
- ☒ **Deployment strategy defined** — see 08_Deployment_Blueprint.md
- ☒ **Rollback strategy defined** — frontend and schema rollback are one-step; data rollback is not automated
- ☒ **Documentation completed** — all ten ODF mandatory deliverables current as of this version
- ☐ **Risks reviewed and accepted by the customer** — pending explicit customer sign-off on the risks in §4

4. Risks, Gaps & Open Issues

#	ITEM	SEVERITY	DETAIL
1	No authenticated identity	High	Role claim is a client-supplied header, not cryptographically verified — see 06_Security_Review_Document.md §1

#	ITEM	SEVERITY	DETAIL
2	Audit trail is client-side only	High	The single domain that never moved server-side; entries live in browser localStorage, are user-editable via developer tools, and carry client-generated timestamps
3	~18 of 44 data models have no write API	Medium	Risk Assessment, Validation, Decision, the Phase 9 Policy domain, the Phase 10 Change domain, and Operations remain client-side only
4	Zero secondary database indexes	Medium	51 relations, no @@index — relation lookups will degrade to sequential scans at volume
5	No automated test suite or CI pipeline	Medium	See 07_Test_Strategy_And_Evidence.md
6	1,132 KB single frontend JS chunk	Medium	No route-level code splitting; grows with every domain added
7	Alerting is not event-driven	Medium	No scheduler, background job, or notification channel — alerts surface only when a user opens the relevant page
8	Single-tenant deployment model	Low–Medium	Multi-tenancy is not implemented
9	Regulatory content is mechanism-only	Low	ISO 42001, EU AI Act, NIST AI RMF, and an expanded RBI mapping are configurable through the Regulatory Knowledge Engine and the Governance Intelligence Studio, but no content for them is pre-loaded
10	Minor dead code and localStorage key versioning debt	Low	~443 lines of unreferenced code; localStorage keys are version-suffixed without a migration path for stale payloads

None of these are hidden defects discovered late — each is a direct, traceable consequence of scope choices documented release-by-release in `docs/governance/`, not a surprise found during this assessment.

5. Recommended Remediation Sequence

PRIORITY	ITEM
P0	Server-side authentication (signed session/JWT, credential verification)
P0	Move audit log writes to the API, with server-issued timestamps and database-level append-only enforcement
P1	Extend the write API to the remaining ~18 data models
P1	Add database indexes on every foreign key and common filter column
P1	Stand up a unit test suite (reasoning engines first) and a CI pipeline
P2	Route-level code splitting to reduce the frontend entry chunk
P2	Remove dead code and externalise seed data from the shipped bundle

6. Readiness Rating

Overall rating: Fair — feature-complete for a single-tenant, trusted-operator deployment; not yet ready for untrusted multi-user production without completing the P0 items above.

This assessment should be read alongside 10_Project_Closure_Report.md for what has been delivered, and 24_Final_Strategic_Blueprint_Design_Partner_Program.md for how the remaining gaps map to the platform's forward engagement model.

Project Closure Report

Delivery summary by release, known limitations, lessons learned

1. Executive Summary

OMG has completed its Core Platform v1.0 baseline: nine governance domains, fifty-six modules, delivered across ten numbered releases and a final strategic positioning update. The platform is live at <https://orchestrai-omg.web.app>, backed by a NestJS + Prisma + Neon PostgreSQL API for the majority of its governance domains. This report closes the current delivery cycle and hands off to the next phase: Founding Governance Partner engagements and the remediation sequence in 09_Production_Readiness_Assessment.md.

2. Summary of Delivered Capability, by Release

RELEASE	DELIVERED
1	Governance Authority Profile, Human Oversight Classification, Autonomy Classification and Authority Matrix
2	Governance State Model, Reassessment Trigger Framework, Review Schedule & Reauthorization Records
3	Evidence Registry — ownership, traceability, lifecycle and expiry tracking
4	Readiness Foundation, Repository Pattern, CRUD API and the data migration utility
4.1	Persistence completion — Neon genuinely seeded, API-first <code>storageService</code> cache, corrected Tenant Settings
5	Compliance Pack Framework — Pack → Requirement → Control → Evidence, coverage without artificial scores
5.1	Compliance Persistence Alignment — the Compliance Pack Framework moved off local storage onto Neon

RELEASE	DELIVERED
6	Universal Regulatory Knowledge & Obligation Engine — Source → Requirement → Obligation → Control → Evidence
7	Governance Intelligence Engine (Foundation) — Policy → Condition → Violation → Finding → Outcome, every outcome explainable
8	Governance Intelligence Engine (Actions Edition) — Outcome → Recommended Action, human Accept/Reject/Defer
9	Governance Decision Traceability Engine — full chain reconstruction, Decision Evidence Pack
10	Governance Intelligence Studio — Condition/Outcome/Action Designers, Rule Mapping Engine, Compliance Pack Builder foundation, Customer Governance Profiles
Final Positioning	"Built Once. Configured Together. Governed Continuously." — Core Platform v1.0 Complete, Founding Governance Partners Program

Full per-release detail, design rationale and validation evidence:

`docs/governance/12` through `24` .

3. Mandatory ODF Governance Artefacts Checklist

- 01_Executive_Solution_Blueprint.md
- 02_Functional_Requirements_Specification.md
- 03_Solution_Architecture_Blueprint.md
- 04_Database_Design_Document.md
- 05_API_Design_Specification.md
- 06_Security_Review_Document.md
- 07_Test_Strategy_And_Evidence.md
- 08_Deployment_Blueprint.md
- 09_Production_Readiness_Assessment.md
- 10_Project_Closure_Report.md (this document)

All ten are current as of Core Platform v1.0. Supplementary (non-mandatory) documents — the per-release addenda (12–24) and the CEAL future-evolution

discussion draft (11) — are maintained alongside them.

4. Known Limitations

Restated in full, not summarised away, from
09_Production_Readiness_Assessment.md:

- No cryptographically verified authentication — the role claim is a client-supplied header.
 - The audit log is the one domain that never moved server-side; it remains browser-localStorage-only.
 - Roughly 18 of 44 data models have no write API (Risk Assessment, Validation, Decision, the earlier Policy and Change domains, Operations).
 - Zero secondary database indexes.
 - No automated test suite or CI pipeline; verification to date has been manual and browser-based.
 - Alerting is computed on read, with no scheduler or notification channel.
 - Single-tenant deployment model; multi-tenancy is not implemented.
 - Regulatory content (ISO 42001, EU AI Act, NIST AI RMF, an expanded RBI mapping) is configurable through the platform but not pre-loaded.
-

5. Future Enhancements

Per 24_Final_Strategic_Blueprint_Design_Partner_Program.md, future work is scoped as configuration and partnership, not platform redesign:

1. **Server-side authentication** and moving the audit log to the API (P0 remediation).
2. **Extending write coverage** to the remaining data models (P1).
3. **Founding Governance Partner engagements** across Banking, NBFC, Insurance, Healthcare, Government and Enterprise — a four-phase model (Governance Discovery → Configuration & Extension → Joint Validation & QA → Production Deployment).

4. **Compliance accelerators** delivered as Governance Intelligence Studio configuration: RBI Guidance, ISO/IEC 42001, EU AI Act, NIST AI RMF, internal enterprise policies, and customer-specific governance controls.
 5. **Operational hardening**: database indexing, a unit test suite and CI pipeline, and route-level code splitting.
-

6. Lessons Learned

- **"Computed, not stored" scaled well.** Deriving Conditions, Outcomes, and Decision Traces live from already-persisted data, rather than caching a duplicate copy, meant Releases 7–9 could each build on the prior release's reasoning without introducing a new place for drift to occur.
 - **The Repository Pattern paid for itself.** Establishing an Api-first, cache-then-network pattern at Release 6 meant every subsequent domain (Compliance, Regulatory Knowledge, Governance Intelligence, Actions, the Studio) followed the same wiring — no domain since has needed the "move off local storage" correction that Release 5.1 had to make for Release 5.
 - **The audit log is the counter-example worth naming.** It is the one domain that predates the Repository Pattern and was never revisited — a concrete illustration of why "everything new follows the pattern" is not the same as "everything is covered," and why this closure report states that gap explicitly rather than letting the pattern's general success imply it.
 - **Configuration boundaries need to be drawn deliberately.** Release 10's Governance Intelligence Studio could have become an unbounded rules engine; scoping it to "detection mechanisms and evaluation order stay code, only which of them are active is configurable" kept it shippable and testable.
 - **Honest gap-tracking is itself a governance deliverable.** Every ODF document in this set states what is not yet built alongside what is — consistent with the platform's own stated purpose of proving governance posture rather than merely claiming it.
-

7. Closure Statement

Core Platform v1.0 is complete against the scope defined in 01_Executive_Solution_Blueprint.md. The project is ready to proceed to Founding Governance Partner engagement and the P0/P1 remediation sequence in 09_Production_Readiness_Assessment.md.

OrchestrAI Model Governance (OMG) — Governance Documentation Package, generated from the mandatory ODF deliverables maintained in the platform repository ([docs/governance/](#)). Every figure is sourced from the live codebase at the time of writing.